

ECON1059 Seminar 4

Notes for the Seminar Series on Python Programming for Economics (ECON1059, Fall 2025)

Juncheng ZHANG, PhD Candidate in Economics, UNNC.

Research interest: Behavioral and Experimental Economics.

Email: Juncheng.ZHANG@nottingham.edu.cn

Importing Modules

random

- If your code uses a function from a module, **always import it first**.
- `import random`
Imports the entire module. Call functions with a prefix: `random.randint(1,10)` .
- `import random as ra`
Same as above, but gives a shorter alias: `ra.randint(1,10)` .

```
In [1]: ## Example 1
import random
print(random.randint(1,10))
```

7

Random Module Summary

- `random.choice(sequence)`
Selects **one random element** from a **non-empty sequence** (e.g., list, tuple, string).
- `random.random()`
Returns a random **float** in the range **[0.0, 1.0)** (includes `0.0` but **excludes** `1.0`).
- `random.uniform(a, b)`
Returns a random **float** in the range **[a, b]** (both endpoints **may be included**, and works even if `a > b`).
- `random.randint(a, b)`
Returns a random **integer** in the **closed interval [a, b]** (both `a` and `b` are included).
- `random.randrange(start, stop, step)`
Returns a random **integer** from a range, including `start` but **excluding** `stop` .

- `random.gauss(mu, sigma)`
Returns a random float drawn from a **normal (Gaussian) distribution** with **mean = mu** and **standard deviation = sigma**.
- `random.seed(value)`
Sets the **seed** for the random number generator to ensure **reproducible results**.
In Jupyter Notebook, the seed should be placed in the **same cell** as the random function.
Without setting a seed, random results will **change on each run**.

- Have a look at the documentation for the package `random` : <https://docs.python.org/3/library/random.html>
 - explore at least two different functions with simple examples.

```
In [2]: # Meal choice
import random
meal = ['McDonald's', 'Pizza Hut', 'KFC', 'Burger King']
print(f"My meal is {random.choice(meal)}")
```

My meal is McDonald's

```
In [3]: # Get a random value from a uniform distribution [a,b)
x = random.uniform(0, 1) # Equivalent to random.random()
print(f"A draw from uniform distribution on [0,1) resulted in: {x: 0.4f}")
```

A draw from uniform distribution on [0,1) resulted in: 0.0677

```
In [4]: ## Example 2 Use random.uniform() in an if statement
if random.uniform(0, 1) > .5:
    print('HELLO')
else:
    print('hello')
```

hello

```
In [5]: # Canteen choices
print(f"Today I am going to canteen {random.randint(1, 4)}")
```

Today I am going to canteen 2

```
In [6]: # Random integer from range [1, 5)
print(random.randrange(1,5))
```

1

```
In [7]: # Seed
random.seed(123456)
print(random.uniform(0, 1))
```

0.8056271362589

`math`

Function / Constant	Meaning	Example (with output)
<code>pi</code>	Pi (≈ 3.14159)	<code>print(math.pi)</code> → 3.141592653589793

Function / Constant	Meaning	Example (with output)
<code>e</code>	Euler's number (≈ 2.71828)	<code>print(math.e)</code> → 2.718281828459045
<code>sqrt(x)</code>	Square root of x	<code>print(math.sqrt(16))</code> → 4.0
<code>exp(x)</code>	e to the power of x	<code>print(math.exp(1))</code> → 2.718281828459045
<code>log(x, base)</code>	Logarithm (default: natural log)	<code>print(math.log(8, 2))</code> → 3.0
<code>sin(x)</code>	Sine (x in radians)	<code>print(math.sin(math.pi/2))</code> → 1.0
<code>cos(x)</code>	Cosine (x in radians)	<code>print(math.cos(0))</code> → 1.0
<code>tan(x)</code>	Tangent (x in radians)	<code>print(math.tan(math.pi/4))</code> → 1.0
<code>fabs(x)</code>	Absolute value (float)	<code>print(math.fabs(-5))</code> → 5.0

Note: You can use `abs()` directly in Python — no need to `import math`.

```
In [8]: ## Example 3 – abs() vs math.fabs()
import math
print(abs(-5))      # → 5      (built-in absolute value)
print(math.fabs(-5)) # → 5.0    (float absolute value from math)
```

5
5.0

matplotlib

- `import matplotlib.pyplot as plt`
Imports the plotting interface.
- `fig, ax = plt.subplots()`
Creates a Figure (`fig`) containing one Axes (`ax`).
- Common plotting methods (called on `ax`)
 - Line: `ax.plot(x, y)`
 - Scatter: `ax.scatter(x, y)`
 - Bar: `ax.bar(x, height)`
 - Pie: `ax.pie(sizes, labels=..., autopct=...)`
- Labels, title, legend
 - `ax.set_xlabel("text")`
 - `ax.set_ylabel("text")`
 - `ax.set_title("text")`
 - `ax.legend()`
- Layout & display
 - `plt.tight_layout()` — avoids overlapping when multiple subplots exist
 - `plt.show()` — displays the final figure

```
In [9]: ## Example 4 Simple Line
import matplotlib.pyplot as plt

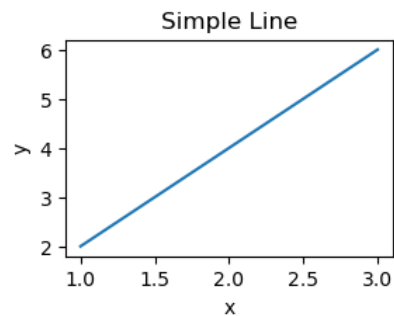
# Create figure and axes
# figsize=(3, 2) means 3 inches wide × 2 inches tall. Default size(6.4, 4.8)
fig, ax = plt.subplots(figsize=(3, 2))

# Data
x = [1, 2, 3]
y = [2, 4, 6]

# Draw a line on this axes
ax.plot(x, y)

# Add labels
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Simple Line")

# Show figure
plt.show()
```



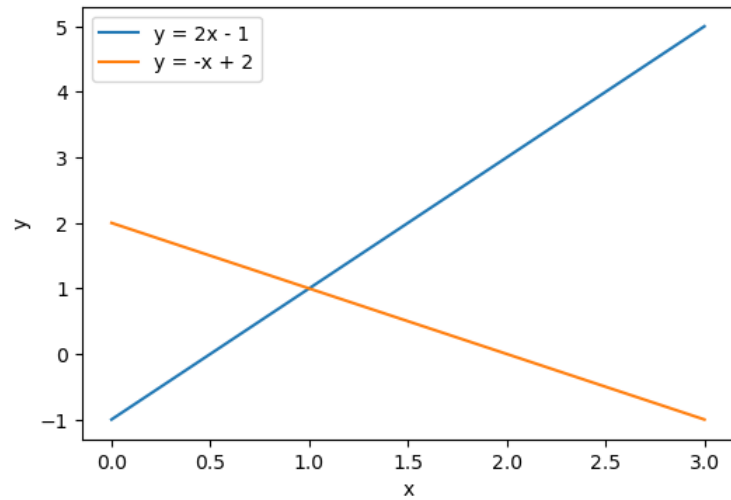
- Graph two lines of the form $y = a \cdot x + b$ on the interval $[0, 3]$, with (1) $a = 2, b = -1$, and (2) $a = -1, b = 2$.

Side question: how many datapoints are necessary to generate this graph?

```
In [10]: import matplotlib.pyplot as plt
# data
data_x = [0, 3] # We only need two datapoints.
data_y_1 = []
data_y_2 = []
for x in data_x:
    data_y_1.append(2*x - 1)
    data_y_2.append(-x + 2)
```

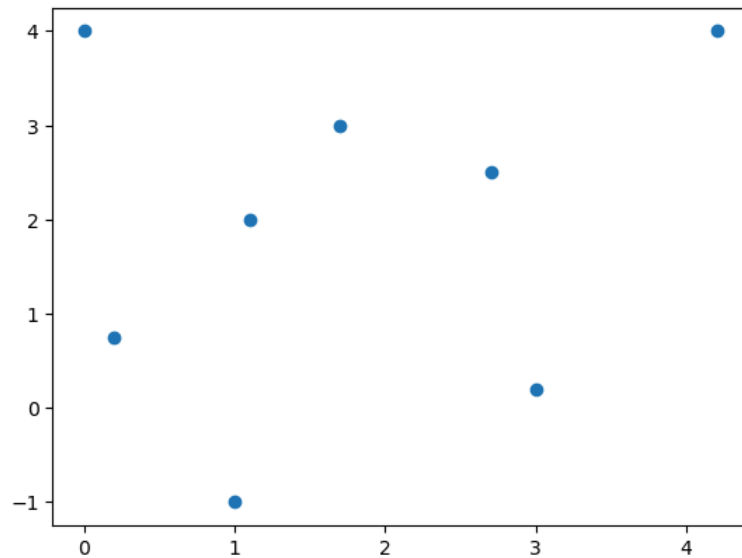
```
In [11]: # figure
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(data_x, data_y_1, label="y = 2x - 1")
ax.plot(data_x, data_y_2, label="y = -x + 2")
ax.set_xlabel("x")
```

```
ax.set_ylabel("y")
ax.legend()
plt.show()
```



- Create any scatterplot graph of your choosing using `ax.scatter(x, y)`.

```
In [12]: import matplotlib.pyplot as plt
# Creating some arbitrary dataset by hand:
x = [0, 1, 1.1, 1.7, 3, 4.2, 2.7, 0.2]
y = [4, -1, 2, 3, 0.2, 4, 2.5, 0.75]
fig, ax = plt.subplots()
ax.scatter(x, y)
plt.show()
```



- Create a scatterplot graph for data generated by $y = x/2 - 2 + e$, where e is a random error that follows a uniform distribution $U[-1, 1]$. Your graph should include 30 datapoints on the interval $[0, 10]$.

Comments on Solution

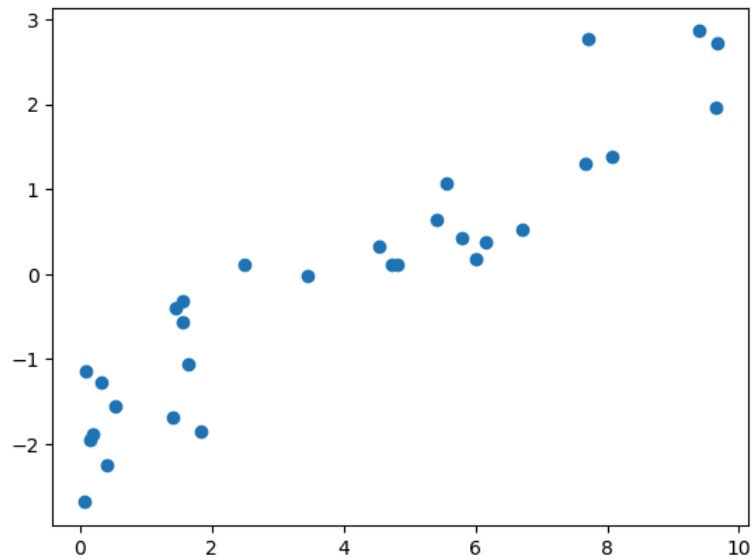
- As the question does not say anything about how x is generated, it makes sense to also generate it randomly (but having 30 equally spaced datapoints is ok too).
- Aside: the following code is a great starting point to generate data for regressions (feel free to replace the uniform error with a normally distributed error). You can even use it to create hypothetical data to pre-test the analysis of an upcoming experiment that you designed (to make sure your experiment is well designed, and your analysis code works)

```
In [13]: import matplotlib.pyplot as plt
import random

random.seed(1234)
data_x = []
data_y = []

for i in range(30):
    #x = i * (10 / 29)          # evenly spaced between 0 and 10
    x = random.uniform(0, 10)
    y = x/2 - 2 + random.uniform(-1, 1)
    data_x.append(x)
    data_y.append(y)
```

```
In [14]: fig, ax = plt.subplots()
ax.scatter(data_x, data_y)
plt.show()
```



- `random.gauss(mu, sigma)`

Returns a random float drawn from a **normal (Gaussian) distribution** with **mean** = `mu` and **standard deviation** = `sigma`.

```
In [15]: ## Example 5 200-Point Scatter:  $x \in [-5, 5]$ ,  $y \sim N(0, 1)$ 
import matplotlib.pyplot as plt
import random

# Start from empty lists
x_values = []
y_values = []

# Generate 200 points
for i in range(200):
    # even spacing from -5 to 5
    x = -5 + i * (10 / 199)      # 10 = width of interval; 199 = steps
    y = random.gauss(0, 1)      # Gaussian noise  $N(0,1)$ 

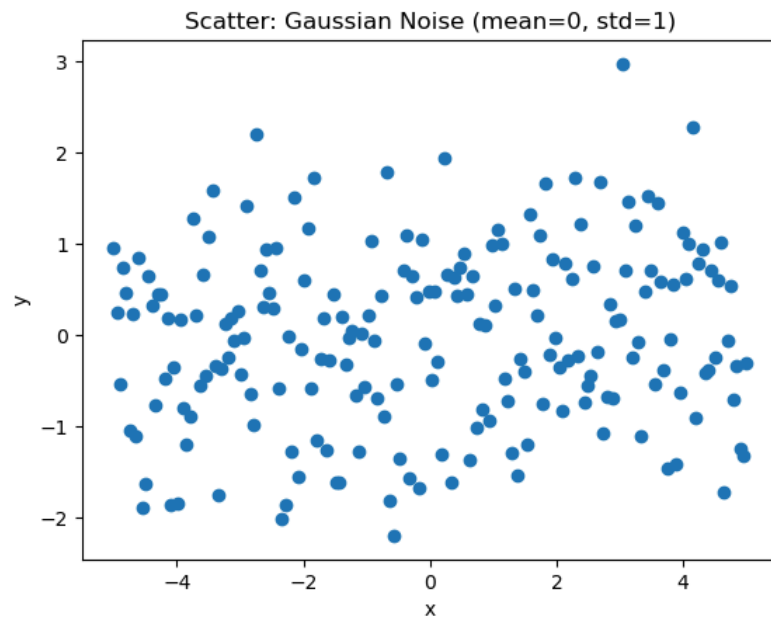
    x_values.append(x)
    y_values.append(y)

# Create figure and axis
fig, ax = plt.subplots()

# Scatter plot
ax.scatter(x_values, y_values)

# Labels
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Scatter: Gaussian Noise (mean=0, std=1)")
```

```
# Show  
plt.show()
```



```
In [16]: ## Example 6 Coin Flip Counts (Bar chart & Pie chart)
import random
import matplotlib.pyplot as plt

heads = 0
tails = 0

# Flip the coin 10,000 times
for i in range(10000):
    # random.choice() selects one element uniformly at random
    flip = random.choice(["Heads", "Tails"])
    if flip == "Heads":
        heads += 1
    else:
        tails += 1

# Print the counts
print("Heads:", heads)
print("Tails:", tails)
```

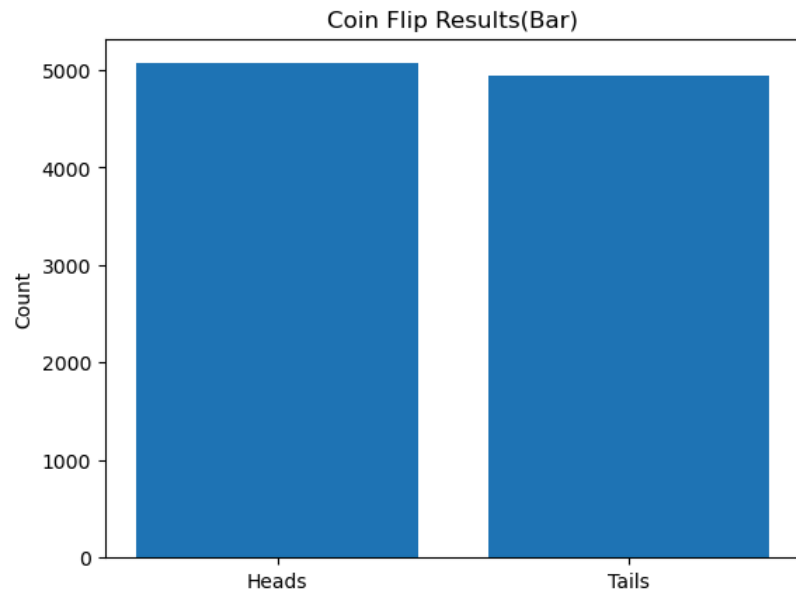
Heads: 5066
Tails: 4934

```
In [17]: # Data
labels = ["Heads", "Tails"]
values = [heads, tails]
```

```
# Create plot (bar)
fig, ax = plt.subplots()
ax.bar(labels, values)

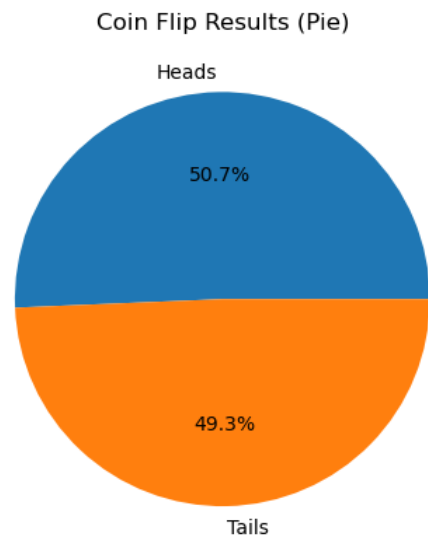
# Labels
ax.set_ylabel("Count")
ax.set_title("Coin Flip Results(Bar)")

plt.show()
```



```
In [18]: # Create plot (pie)
fig, ax = plt.subplots()
ax.pie(values, labels=labels, autopct="%1.1f%")

ax.set_title("Coin Flip Results (Pie)")
plt.show()
```



- `fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(6,4))`

Creates a Figure (`fig`) with a grid of Axes (`ax`).

- `nrows` : number of rows (default `1`)
- `ncols` : number of columns (default `1`)
- `figsize` : figure size in inches (default `~ (6.4, 4.8)`)

Example:

- `plt.subplots(1,2)` → 1 row, 2 columns (2 subplots)
- `plt.subplots(2,2)` → 2×2 grid (4 subplots)

```
In [19]: ## Example 7 Combine

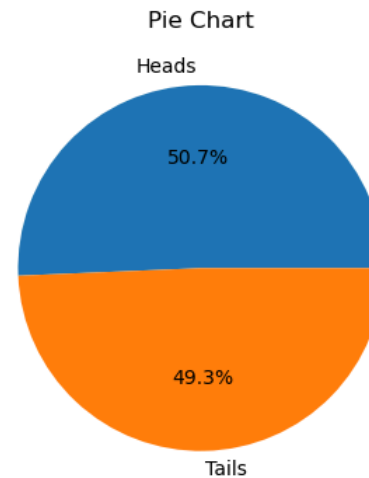
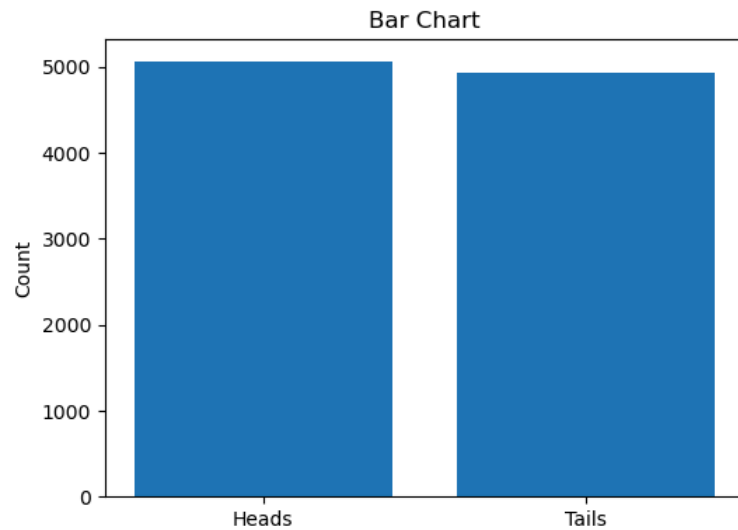
# Data
labels = ["Heads", "Tails"]
values = [heads, tails]

# Create a figure with 1 row, 2 columns
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

# Bar chart
ax1.bar(labels, values)
ax1.set_title("Bar Chart")
ax1.set_ylabel("Count")

# Pie chart
ax2.pie(values, labels=labels, autopct="%1.1f%%")
ax2.set_title("Pie Chart")

plt.tight_layout()
plt.show()
```



Simulate Random Events with random()

- Coinflipping. Flip a 'coin' n-times using the random integer function. What is the frequency (percentage) of heads in your sample?

- Hint 1
- Hint 2
- Hint 2 continued

```
In [20]: # Solution 1: most basic implementation.
import random

def calculate_heads_n_coinflips(num_coinflips):
    num_heads = 0
    for i in range(num_coinflips):
        x = random.randint(0,1)
        num_heads = num_heads + x

    avg = num_heads / num_coinflips
    return avg

print(f"{calculate_heads_n_coinflips(10000): 0.3f}")

0.506
```

```
In [21]: # Solution 2: tracks all coinflips before calculating average.
import random # To keep examples self-contained, I import again.

def flip_coin_n_times(num_coinflips):
    data = []
    for i in range(num_coinflips):
        data.append(random.randint(0,1))
    return data
```

```
total_flips = 10000
coinflips = flip_coin_n_times(total_flips)

# Using build-in function sum(). Could you write it yourself?
avg_heads = sum(coinflips)/total_flips
print(avg_heads)
```

0.505

- Neither Solution 1 or Solution 2 is inherently better than the other
- Solution 1 is probably more to the point of the question.
- Solution 2 is a more general approach to these kind of questions, which could be useful in other settings or to solve more general problems. However, it is less memory efficient

"Guess a Number" Game

- Create a game where your aim is to guess a number (among 1,...,10) that the computer secretly selected (at random) within 3 tries.
- Note: recall that input() returns a string. Turn it into an integer with int().
- Make sure to test your game properly! In this case, you may either want to print() the secret number or get rid of the randomness for a while

Comments on Solution

- The key idea in the code below is to have a while loop that repeats the relevant if-clauses until the game is over.
- The official solution relies on a very simple implementation, and is far from ideal. There is too much code-repetition. Maybe you can try and improve it (in a way that is still easily readable)
- Another nice extension you could add is to track previous guesses
- Jupyter notebook will run this code until you win/lose. To stop early, simply interrupt Kernel and start over
- Solution may be difficult to read in html-slides. Please see pdf or original code in that case.

```
In [22]: import random

def guess_a_number_game():
    total_misses_allowed = 3 # Set "difficulty"
    secret_number = random.randint(1,10)

    total_misses = 0 # Do not set.

    while total_misses < total_misses_allowed:
        # The strict inequality is correct. Try to see for yourself

        print(f"You have {total_misses_allowed - total_misses} more guesses.")
        guess = int(input(f"Please make a guess."))

        if guess == secret_number:
            print("\nWah, you're a genius. You won!")
            break # Not ideal. Maybe you can write this better
        elif total_misses < total_misses_allowed - 1:
            print(f"\nNope, {guess} is not correct.\n")
            total_misses += 1
        else:
            print(f"\nNope, {guess} is not correct. \n")
            print("GameOver")
            total_misses += 1
```

```
guess_a_number_game() # Run game
```

You have 3 more guesses).

Nope, 3 is not correct.

You have 2 more guesses).

Nope, 2 is not correct.

You have 1 more guesses).

Nope, 1 is not correct.

GameOver

Scope

- Define a function `h()` inside another function `f()`. Can you call `h()` inside `f()`? Can you call it outside of `f()`? What does this tell you about scope?

```
In [23]: def outside_f():
         def inside_f():
             print("Guten Tag means good day in German : ")

         for i in range(3):
             inside_f()
```

```
In [24]: outside_f()
```

```
Guten Tag means good day in German : )
Guten Tag means good day in German : )
Guten Tag means good day in German : )
```

```
In [25]: # Calling inside_f() outside of its scope doesn't work.
         inside_f()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[25], line 2
      1 # Calling inside_f() outside of its scope doesn't work.
----> 2 inside_f()

NameError: name 'inside_f' is not defined
```

Takeaway. Not only are variables defined inside a function local, but so are other named objects.